VSP TECHVERSE
An Institute of AI & Robotics

Python (Pandas) for AI & Robotics

Load, clean, explore and plot data — the skills behind every AI model



VSP TECHVERSE
An Institute of AI & Robotics

Pandas HOME

- Pandas is a Python library for working with data sets
- Analyse, clean, explore and manipulate data
- Built around the DataFrame — a table of rows and columns

example.py

```
import pandas as pd

data = {"robot": ["Arm-1", "Rover-2", "Drone-3"],
        "battery": [82, 64, 91]}

df = pd.DataFrame(data)
print(df)
```

OUTPUT

```
      robot  battery
0     Arm-1       82
1   Rover-2       64
2   Drone-3       91
```

AI & ROBOTICS USE

Turn raw robot telemetry into tidy tables you can query.

Pandas Intro

- Name refers to both "Panel Data" and "Python Data Analysis"
- Created by Wes McKinney in 2008, built on NumPy
- Handles big data sets and makes them readable and relevant

example.py

```
import pandas as pd

df = pd.DataFrame({"speed": [1.2, 0.8, 1.5, 1.1]})

print(df["speed"].mean())
print(df.describe().loc[["min", "max"]])
```

OUTPUT

```
1.15
     speed
min    0.8
max    1.5
```

AI & ROBOTICS USE

Spot patterns in sensor logs before training a model.

Pandas Getting Started

- Install with pip install pandas
- Import it with the alias pd
- Check the installed version in __version__

example.py

```
# terminal
pip install pandas

import pandas as pd

print(pd.__version__)
```

OUTPUT

```
3.0.5
```

AI & ROBOTICS USE

Runs the same on a laptop, a cloud notebook or a robot's onboard computer.

Pandas Series

- A Series is one column — a labelled 1-D array
- Default labels are 0, 1, 2 … ; set your own with index
- Can also be created from a dictionary

example.py

```
import pandas as pd

a = [1, 7, 2]

s = pd.Series(a)
print(s)

s2 = pd.Series(a, index=["x", "y", "z"])
print(s2["y"])
```

OUTPUT

```
0    1
1    7
2    2
dtype: int64
7
```

AI & ROBOTICS USE

One Series per sensor channel: battery, speed, temperature.

Pandas DataFrames

- A DataFrame is a 2-D table: rows and columns
- loc[] returns one row or a list of rows
- Each column is a Series

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data)

print(df.loc[0])
print(df.loc[[0, 1]])
```

OUTPUT

```
battery    82.0
speed       1.2
Name: 0, dtype: float64
   battery  speed
0       82    1.2
1       64    0.8
```

AI & ROBOTICS USE

Each row is one robot (or one time step); each column a measurement.

Pandas Read CSV

- CSV files store plain-text tables
- pd.read_csv() loads one into a DataFrame
- to_string() prints every row

example.py

```
import pandas as pd

# data.csv: Robot, Battery, Speed
df = pd.read_csv("data.csv")

print(df.to_string())
```

OUTPUT

```
     Robot  Battery  Speed
0    Arm-1       82    1.2
1  Rover-2       64    0.8
2  Drone-3       91    2.5
3    Arm-4       77    1.1
```

AI & ROBOTICS USE

Load logged telemetry exported from a robot controller.

Pandas Read JSON

- JSON is the standard format for web APIs
- pd.read_json() loads a JSON file
- A Python dictionary can become a DataFrame too

example.py

```
import pandas as pd

# data.json: [{"Robot": "Arm-1", ...}, ...]
df = pd.read_json("data.json")

print(df.to_string())
```

OUTPUT

```
     Robot  Battery
0    Arm-1       82
1  Rover-2       64
2  Drone-3       91
```

AI & ROBOTICS USE

Pull robot status from a cloud API response.

Pandas Analyzing Data

- head(n) shows the first rows, tail(n) the last
- shape gives (rows, columns)
- info() lists column types and non-null counts

example.py

```
import pandas as pd

df = pd.read_csv("data.csv")

print(df.head(2))
print(df.tail(2))
print(df.shape)
```

OUTPUT

```
     Robot  Battery  Speed
0    Arm-1       82    1.2
1  Rover-2       64    0.8
     Robot  Battery  Speed
2  Drone-3       91    2.5
3    Arm-4       77    1.1
(4, 3)
```

AI & ROBOTICS USE

Quickly check a new dataset before building on it.

DataFrame Slicing

- Slicing selects part of a DataFrame — rows, columns or both
- iloc[] slices by position (integer-based indexing)
- loc[] slices by label (index labels or column labels)

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data, index=list("ABC"))

print(df.iloc[0:2])          # by position
print(df.loc["A":"B"])       # by label
```

OUTPUT

```
   battery  speed
A       82    1.2
B       64    0.8
   battery  speed
A       82    1.2
B       64    0.8
```

AI & ROBOTICS USE

Cut a robot log down to one run, one joint or one time window.

Slicing with iloc[]

- iloc[row, column] uses integer positions from 0
- The end of a position slice is NOT included
- Pass a list to pick specific rows or columns

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data, index=list("ABC"))

print(df.iloc[0:2])
print(df.iloc[1, 0])
print(df.iloc[[0, 2], [0, 1]])
```

OUTPUT

```
   battery  speed
A       82    1.2
B       64    0.8
64
   battery  speed
A       82    1.2
C       91    2.5
```

AI & ROBOTICS USE

Grab the first N frames or the last reading by position.

Slicing with loc[]

- loc[row, column] uses index and column labels
- The end label of a loc slice IS included
- Works with conditions: df.loc[df["battery"] > 70]

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data, index=list("ABC"))

print(df.loc["A":"B"])
print(df.loc["B", "battery"])
print(df.loc[df["battery"] > 70])
```

OUTPUT

```
   battery  speed
A       82    1.2
B       64    0.8
64
   battery  speed
A       82    1.2
C       91    2.5
```

AI & ROBOTICS USE

Select a robot by name, or every unit above a battery level.

Renaming Columns and Rows

- rename() changes column or row labels
- columns={old: new} renames columns; index={old: new} renames rows
- Returns a new DataFrame — assign it back

example.py

```python
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data)

df = df.rename(columns={"battery": "battery_pct"})
df = df.rename(index={0: "Arm-1", 1: "Rover-2"})
print(df)
```

OUTPUT

```
         battery_pct  speed
Arm-1             82    1.2
Rover-2           64    0.8
2                 91    2.5
```

AI & ROBOTICS USE

Give raw sensor channels clear names like battery_pct.

Adding or Inserting Columns

- df["new"] = values adds a column at the end
- insert(position, name, values) places it exactly
- A new column can be calculated from existing ones

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data)

df["payload"] = [5, 3, 4]
df.insert(1, "id", ["a", "b", "c"])
df["kmh"] = df["speed"] * 3.6
print(df)
```

OUTPUT

```
   battery id  speed  payload   kmh
0       82  a    1.2        5  4.32
1       64  b    0.8        3  2.88
2       91  c    2.5        4  9.00
```



AI & ROBOTICS USE

Add derived features such as speed in km/h for a model.

Updating Column Values

- Assign to a whole column to replace its values
- loc[] updates only the rows that match a condition
- replace() swaps one value for another

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data)

df["battery"] = df["battery"] + 5
df.loc[df["speed"] > 2, "speed"] = 2.0
df["speed"] = df["speed"].replace(0.8, 1.0)
print(df)
```

OUTPUT

```
   battery  speed
0       87    1.2
1       69    1.0
2       96    2.0
```



AI & ROBOTICS USE

Correct a miscalibrated sensor across the whole dataset.

Removing Unnecessary Columns

- drop(columns=[...]) removes one or more columns
- del df["col"] and pop() also remove a column
- Keep only the features your model needs

example.py

```
import pandas as pd

data = {"battery": [82, 64, 91],
        "speed": [1.2, 0.8, 2.5]}
df = pd.DataFrame(data)

df["payload"] = [5, 3, 4]

df = df.drop(columns=["speed"])
del df["payload"]
print(df)
```

AI & ROBOTICS USE

Drop IDs and unused channels before training.

OUTPUT

```
   battery
0       82
1       64
2       91
```

Cleaning Data

- Bad data gives bad AI — clean it first
- Typical problems: empty cells, wrong format, wrong data, duplicates
- Spot them early with isna() and duplicated()

example.py

```
import pandas as pd

df = pd.DataFrame({
    "battery": [82, None, 91, 91],
    "speed": [1.2, 0.8, 2.5, 2.5]})

print(df.isna().sum())
print(df.duplicated().sum())
```

OUTPUT

```
battery    1
speed      0
dtype: int64
1
```

AI & ROBOTICS USE

Faulty sensor readings can make a robot misjudge its world.

Cleaning Empty Cells

- dropna() removes rows that contain empty cells
- fillna(value) replaces them with a value
- Fill with the mean, median or mode of the column

example.py

```
import pandas as pd

df = pd.DataFrame({"battery": [82, None, 91]})

print(df.dropna())

x = df["battery"].mean()
print(df["battery"].fillna(x))
```

OUTPUT

```
   battery
0     82.0
2     91.0
0    82.0
1    86.5
2    91.0
Name: battery, dtype: float64
```

AI & ROBOTICS USE

Fill gaps from dropped sensor packets instead of losing rows.

Cleaning Wrong Format

- Cells in the wrong format break calculations
- to_datetime() converts text to dates
- to_numeric() converts text to numbers

example.py

```
import pandas as pd

df = pd.DataFrame({
    "date": ["2026/10/01", "2026/10/02"],
    "temp": ["41.5", "39"]})

df["date"] = pd.to_datetime(df["date"])
df["temp"] = pd.to_numeric(df["temp"])
print(df.dtypes)
```

OUTPUT

```
date    datetime64[us]
temp           float64
dtype: object
```

AI & ROBOTICS USE

Timestamps arrive as text — convert them before analysis.

Cleaning Wrong Data

- Wrong data = impossible values (battery of 640 %)
- Fix a value with loc[], or loop and apply a rule
- Or drop the offending rows

example.py

```
import pandas as pd

df = pd.DataFrame({"robot": ["Arm-1", "Rover-2"],
                   "battery": [82, 640]})

for x in df.index:
    if df.loc[x, "battery"] > 100:
        df.loc[x, "battery"] = 100

print(df)
```

OUTPUT

```
     robot  battery
0    Arm-1       82
1  Rover-2      100
```

AI & ROBOTICS USE

Reject impossible readings before they reach the controller.

Removing Duplicates

- duplicated() returns True for repeated rows
- drop_duplicates() removes them
- Repeated logs would bias a trained model

example.py

```
import pandas as pd

df = pd.DataFrame({
    "robot": ["Arm-1", "Rover-2", "Rover-2"],
    "battery": [82, 64, 64]})

print(df.duplicated())
print(df.drop_duplicates())
```

OUTPUT

```
0    False
1    False
2     True
dtype: bool
     robot  battery
0    Arm-1       82
1  Rover-2       64
```

AI & ROBOTICS USE

Remove double-logged sensor frames from a recording.

Pandas Correlations

- corr() measures how columns relate, from -1 to 1
- Near 1: move together · near -1: opposite
- Near 0: no relationship

example.py

```
import pandas as pd

df = pd.DataFrame({
    "speed": [1, 2, 3, 4, 5],
    "drain": [2, 4, 5, 8, 10],
    "payload": [5, 3, 4, 2, 1]})

print(df.corr().round(2))
```

OUTPUT

```
          speed  drain  payload
speed      1.00   0.99    -0.90
drain      0.99   1.00    -0.94
payload   -0.90  -0.94     1.00
```

AI & ROBOTICS USE

Find which sensors actually predict battery drain.

Pandas Plotting

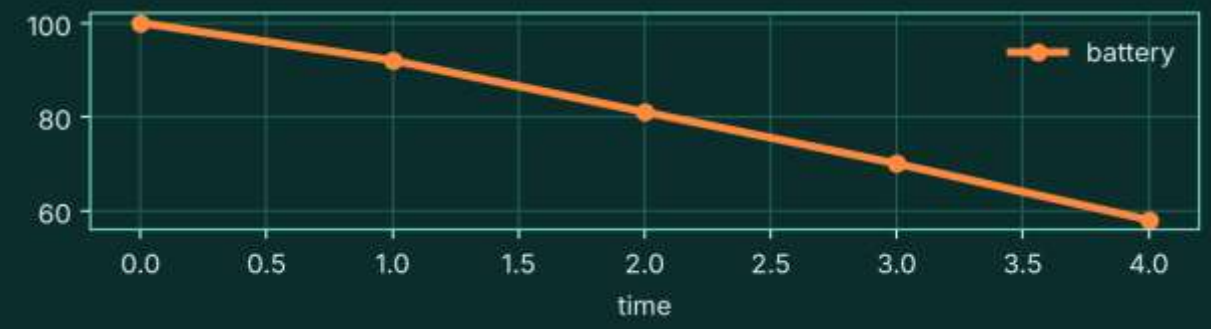
- plot() draws charts straight from a DataFrame
- Needs Matplotlib; call plt.show() to display
- kind= picks line, bar, scatter, hist and more

example.py

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({"time": [0, 1, 2, 3, 4],
                   "battery": [100, 92, 81, 70, 58]})
df.plot(x="time", y="battery")
plt.show()
```

OUTPUT



AI & ROBOTICS USE

Visualise battery drain or sensor drift at a glance.

VSP TECHVERSE
An Institute of AI & Robotics

Thank You

Clean data in, smarter robots out.



VSP TECHVERSE
An Institute of AI & Robotics